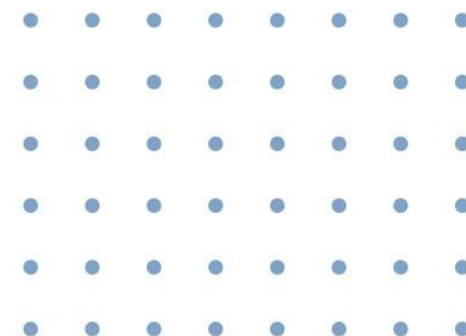
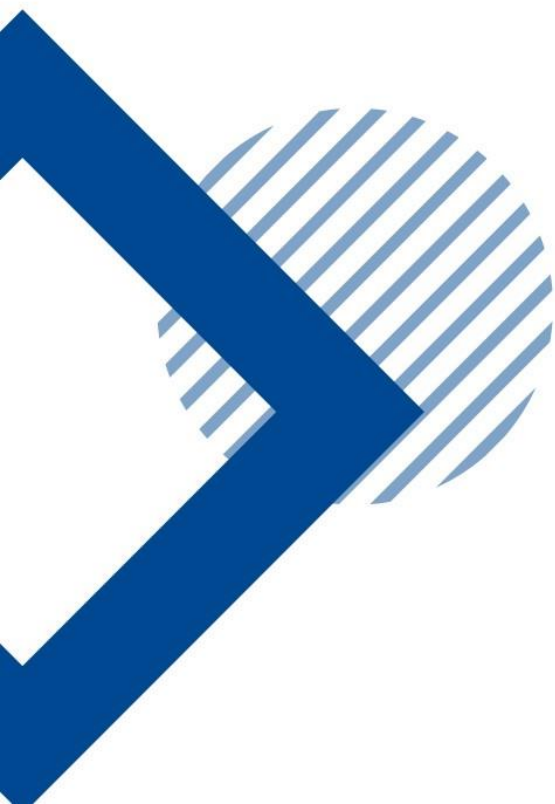


Лекция №3

Управление памятью



Введение

Иерархия памяти: Компьютеры обладают несколькими мегабайтами очень быстрой, дорогой и энергозависимой кэш-памяти, несколькими гигабайтами памяти, средней как по скорости, так и по цене, а также несколькими терабайтами памяти на довольно медленных, сравнительно дешевых дисковых накопителях.

Задача операционной системы – превратить данную иерархии в абстракцию и управлять данной абстракцией.

Часть операционной системы, которая управляет иерархией памяти, называется менеджером, или диспетчером, памяти.

Память без использования абстракций

Простейшая абстракция памяти – отсутствие абстракции.

Универсальные машины до начала 1960-х годов, миникомпьютеры до начала 1970-х годов и персональные компьютеры до начала 1980-х годов НЕ использовали абстракции памяти.

Каждой программе была доступна физическая память. От 0 до некоторого максимального адреса, где каждый адрес соответствовал ячейке памяти, содержащей некоторый набор бит.

При таком устройстве системы памяти процессы могут запускаться только по одному. При вызове команды, операционная система копирует запрошенную программу с диска в память, после чего выполняет эту программу. Когда процесс завершает свою работу, операционная система отображает символ приглашения и ожидает ввода новой команды. По получении команды она загружает в память новую программу, записывая ее поверх первой программы.

Память без использования абстракций



Три простых способа организации памяти при наличии операционной системы и одного пользовательского процесса (существуют также и другие варианты).

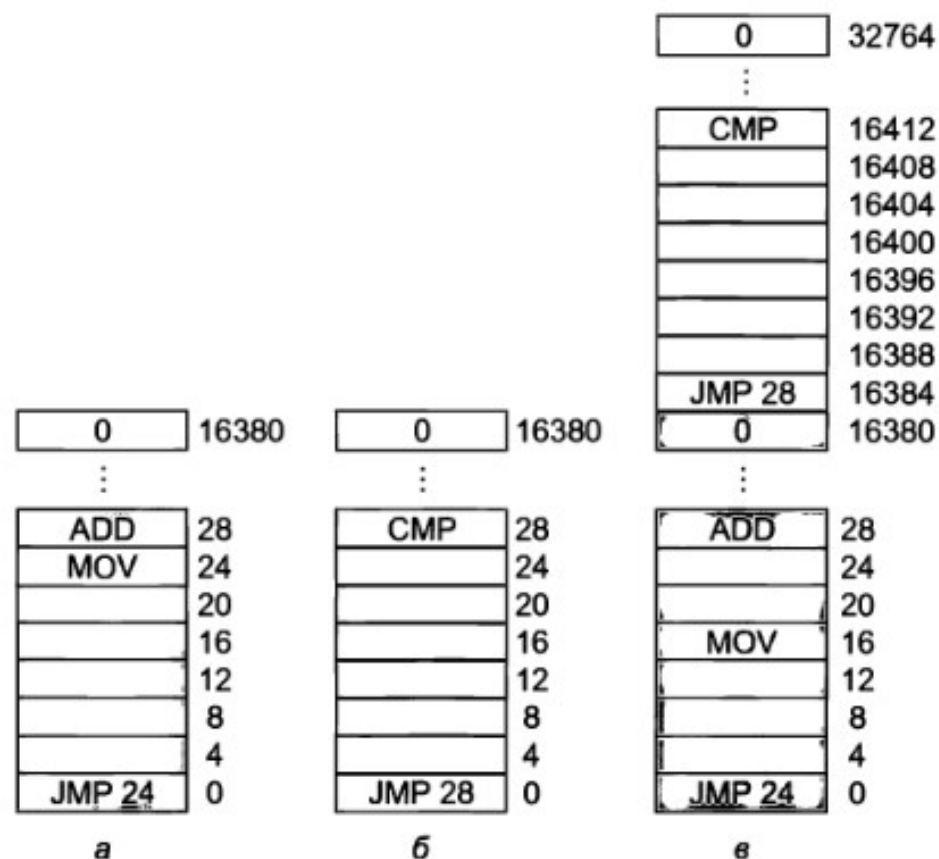
Память без использования абстракций

Предоставление физической памяти процессам имеет ряд серьезных недостатков:

1. Если пользовательские программы могут обращаться к каждому байту памяти, они легко могут преднамеренно или случайно испортить операционную систему. Эта проблема присутствует даже при запуске всего лишь одной пользовательской программы (приложения).
2. При использовании данной модели довольно сложно организовать одновременную (поочередную, если имеется лишь один центральный процессор) работу нескольких программ.

Понятие адресного пространства

Адресное пространство – набор адресов, который может быть использован процессом для обращения к памяти. У каждого процесса имеется свое собственное адресное пространство, независимое от того адресного пространства, которое принадлежит другим процессам (за исключением тех особых обстоятельств, при которых процессам требуется совместное использование их адресных пространств).



Базовый и ограничительный регистры

Классическим решением проблемы трансляции адресов является оснащение каждого центрального процессора двумя специальными аппаратными регистрами: базовым и ограничительным. При использовании этих регистров программы загружаются в последовательно расположенные свободные области памяти без модификации адресов в процессе загрузки.

При запуске процесса в базовый регистр загружается физический адрес, с которого начинается размещение программы в памяти, а в ограничительный регистр загружается длина программы.

При каждой ссылке процесса на память с целью извлечения команды или записи слова данных аппаратура центрального процессора перед выставлением адреса на шине памяти добавляет к адресу, сгенерированному процессом, значение базового регистра. Одновременно аппаратура проверяет, не равен ли предлагаемый адрес значению ограничительного регистра или не превышает ли он это значение, и в этом случае генерируется отказ и доступ прерывается.

Базовый и ограничительный регистры

Использование базового и ограничительного регистров – простой способ предоставления каждому процессу своего собственного закрытого адресного пространства, поскольку к каждому автоматически сгенерированному адресу перед обращением к памяти добавляется значение базового регистра. Многие реализации предусматривают такую защиту базового и ограничительного регистров, при которой изменить их значения может только операционная система.

Недостатком перемещений с использованием базовых и ограничительных регистров является необходимость применения операций сложения и сравнения к каждой ссылке на ячейку памяти. Сравнение может осуществляться довольно быстро, но сложение является слишком медленной операцией из-за затрат времени на вспомогательный сигнал переноса, если не используются специальные сумматоры.

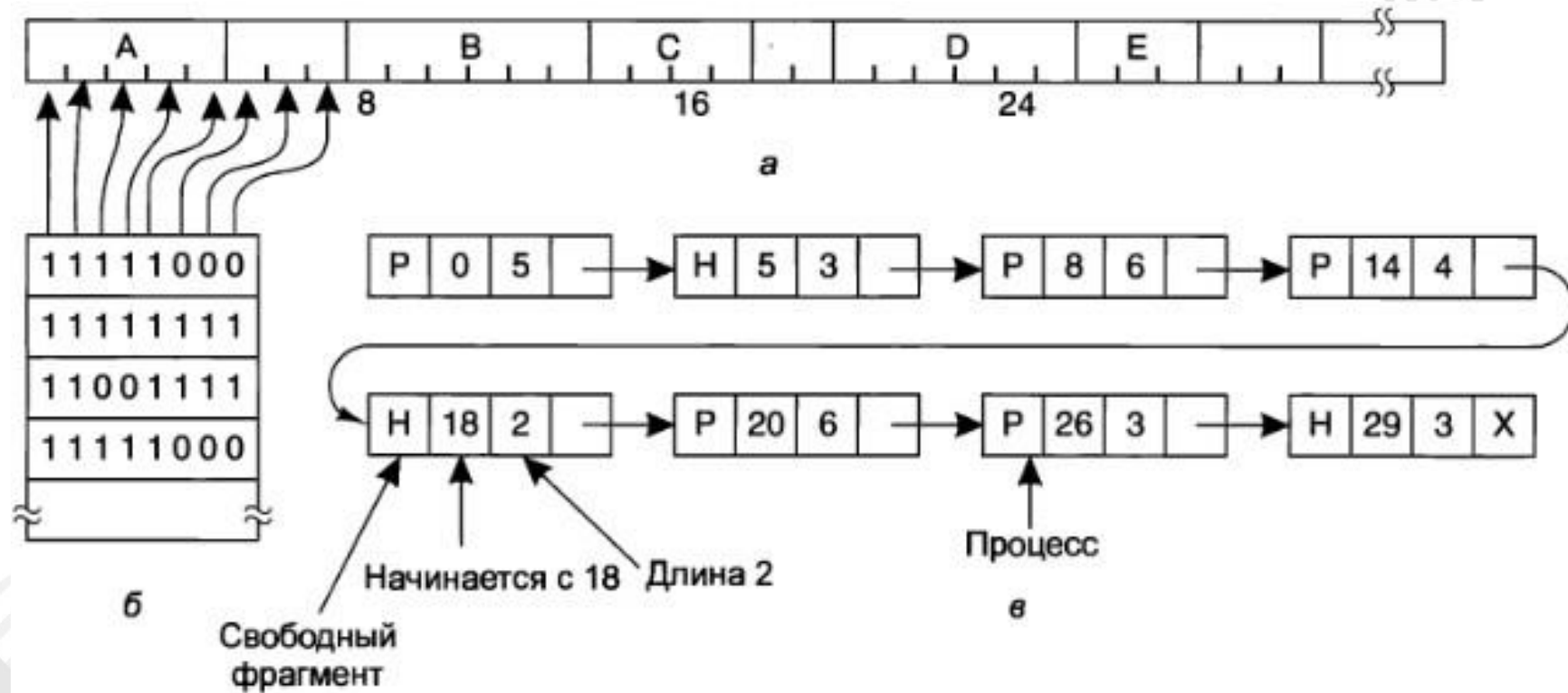
Свопинг

Если компьютер обладает достаточным объемом памяти для размещения всех процессов, то все рассмотренные до сих пор схемы будут в той или иной степени работоспособны. Но на практике суммарный объем оперативной памяти, необходимый для размещения всех процессов, зачастую значительно превышает имеющийся объем ОЗУ.

Свопинг – размещение в памяти всего процесса целиком. Процесс запускается на некоторое время, а затем сбрасывается на диск. Бездействующие процессы большую часть времени хранятся на диске и в нерабочем состоянии не занимают пространство оперативной памяти.

Если предполагается, что большинство процессов по мере выполнения будут разрастаться, необходимо распределять небольшой объем дополнительной памяти при каждой загрузке процесса из области свопинга на диск в память (перемещении процесса внутри памяти), чтобы процессы помещались в отведенной им памяти.

Управление свободной памятью



Часть памяти с пятью процессами и тремя свободными пространствами.

Алгоритмы управления свободной памятью

- первое подходящее. Диспетчер памяти сканирует список сегментов до тех пор, пока не найдет пустое пространство подходящего размера. Затем пустое пространство разбивается на две части: одна для процесса и одна для неиспользуемой памяти;
- следующее подходящее. Данный алгоритм работает так же, как и «первое подходящее», за исключением того, что отслеживает свое местоположение, как только находит подходящее пустое пространство. При следующем вызове для поиска пустого пространства он начинает поиск в списке с того места, на котором остановился в прошлый раз, а не приступает к поиску с самого начала, как в предыдущем алгоритме;
- наиболее подходящее. Данный алгоритм ведет поиск по всему списку от начала до конца и выбирает наименьшее соответствующее пустое пространство. Вместо того чтобы разбивать большое пустое пространство, которое может пригодиться чуть позже, данный алгоритм пытается подыскать пустое пространство, близкое по размеру к необходимому, чтобы наилучшим образом соответствовать запросу и имеющимся пустым пространствам;
- наименее подходящее. Данный алгоритм выбирает самое большое подходящее пустое пространство, чтобы вновь образующееся пустое пространство было достаточно большим для дальнейшего использования;
- быстро искомое подходящее. Его использование предусматривает ведение отдельных списков для некоторых наиболее востребованных искомых размеров.

Оверлеи

Проблема запуска программ, превышающих по объему размер имеющейся памяти, возникла еще на заре компьютерной эры.

В 60-е годы было принято решение разбивать программы на небольшие части, называемые оверлеями. При запуске программы в память загружался только администратор оверлейной загрузки, который тут же загружал и запускал оверлей с порядковым номером 0. Когда этот оверлей завершал свою работу, он мог сообщить администратору загрузки оверлеев о необходимости загрузки оверлея 1. Некоторые оверлейные системы имели довольно сложное устройство, позволяя одновременно находиться в памяти множеству оверлеев. Оверлеи хранились на диске, и их свопинг с диска в память и обратно осуществлялся администратором загрузки оверлеев.

Работа по свопингу оверлеев с диска в память и обратно выполнялась операционной системой, при этом разбиение программ на части выполнялось программистом в ручном режиме. Разбиение больших программ на небольшие модульные части было очень трудоемкой, скучной и не застрахованной от ошибок работой.

Виртуальная память

В основе виртуальной памяти лежит идея, что у каждой программы имеется свое собственное адресное пространство, которое разбивается на участки, называемые страницами.

Каждая страница представляет собой непрерывный диапазон адресов. Эти страницы отображаются на физическую память, но для запуска программы присутствие в памяти всех страниц не обязательно.

Когда программа ссылается на часть своего адресного пространства, **находящегося в физической памяти**, аппаратное обеспечение осуществляет необходимое отображение на лету.

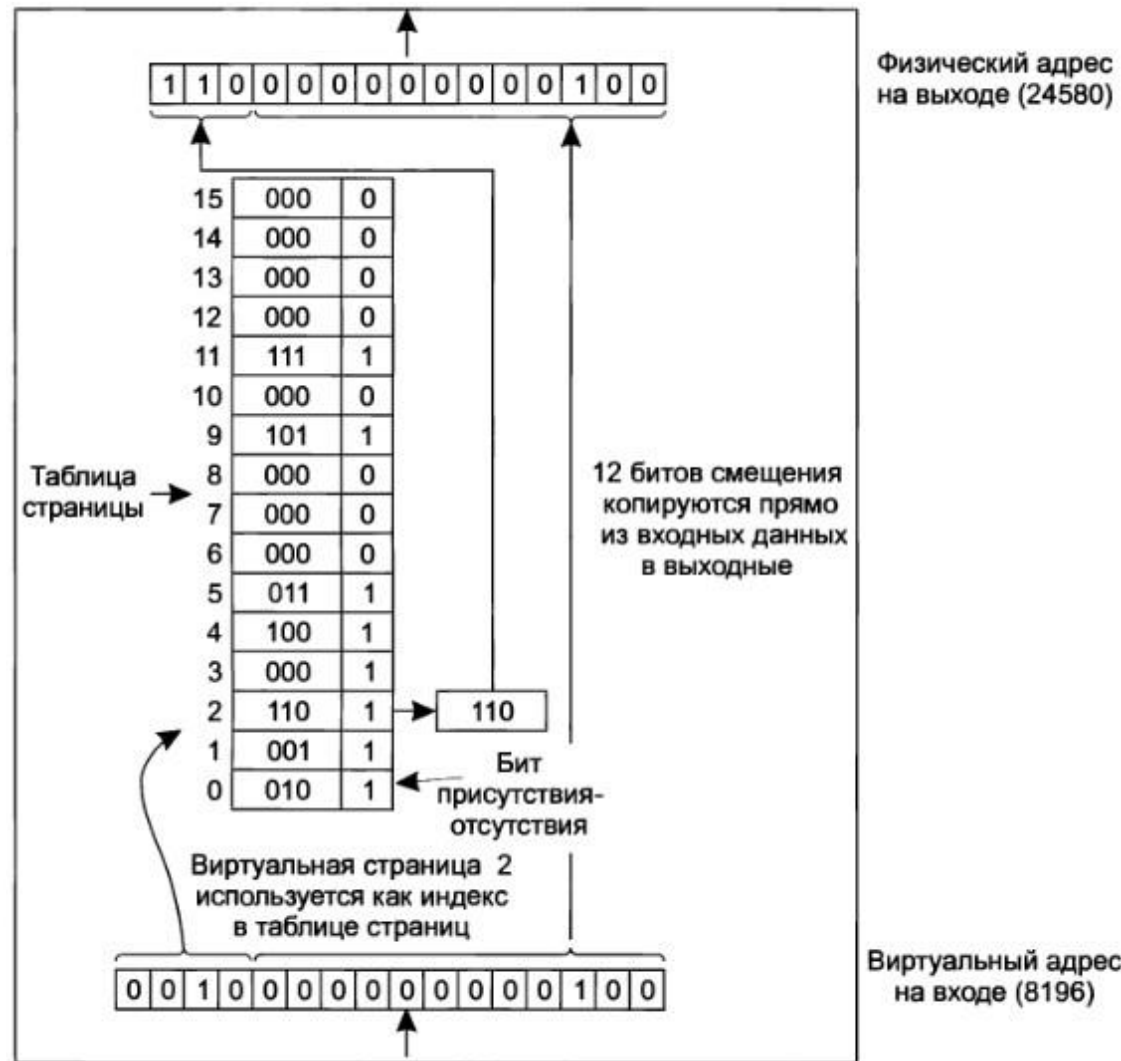
Когда программа ссылается на часть своего адресного пространства, которое **НЕ находится в физической памяти**, операционная система предупреждается о том, что необходимо получить недостающую часть и повторно выполнить потерпевшую неудачу команду.

Страничная организация памяти



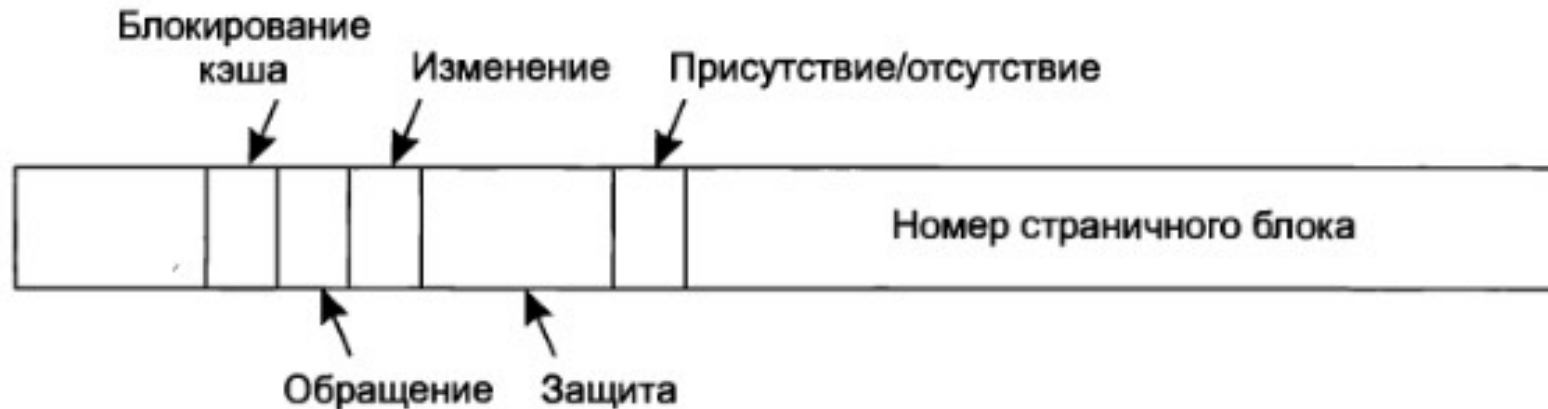
Связь между виртуальными адресами и адресами физической памяти.

Устройство диспетчера памяти



Преобразование диспетчером памяти виртуального адреса в физический для 16 страниц по 4 Кбайт.

Таблица страниц



Предназначение таблицы страниц заключается в отображении виртуальных страниц на страничные блоки.

- бит присутствия-отсутствия – если установлен в 1, запись имеет смысл и может быть использована;
- бит защиты – 0 для чтения-записи и значением 1 только для чтения;
- бит изменения – если 1, страницу необходимо записать на диск;
- бит обращения – необходим для работы алгоритмов замещения страниц;
- бит блокирования кэширования – отключение кэширования.

Особенности страничной организации памяти

1. Буферы быстрого преобразования адреса.
Буфер быстрого преобразования адреса (TLB) – небольшое устройство для отображения виртуальных адресов на физические без просмотра таблицы страниц.
2. Программное управление буфером TLB.
Программное обновление индекса TLB в случае отсутствия страницы в TLB и ее наличии в ОЗУ.
3. Многоуровневые таблицы страниц.
4. Инвертированные таблицы страниц.

Алгоритмы замещения страниц



Алгоритм	Особенности
Оптимальный	Не может быть реализован, но полезен в качестве оценочного критерия
NRU (Not Recently Used) — алгоритм исключения недавно использовавшейся страницы	Является довольно грубым приближением к алгоритму LRU
FIFO (First-In, First-Out) — алгоритм «первой пришла, первой и ушла»	Может выгрузить важные страницы
Алгоритм «второй шанс»	Является существенным усовершенствованием алгоритма FIFO
Алгоритм «часы»	Вполне реализуемый алгоритм
LRU (Least Recently Used) — алгоритм замещения наименее востребованной страницы	Очень хороший, но труднореализуемый во всех тонкостях алгоритм
NFU (Not Frequently Used) — алгоритм нечастого востребования	Является довольно грубым приближением к алгоритму LRU
Алгоритм старения	Вполне эффективный алгоритм, являющийся неплохим приближением к алгоритму LRU
Алгоритм рабочего набора	Весьма затратный для реализации алгоритм
WSClock	Вполне эффективный алгоритм



Сегментация

Для решения многих проблем наличие двух и более отдельных виртуальных адресных пространств может быть более рациональным вариантом, чем наличие только одного адресного пространства.

Например, у компилятора имеется множество таблиц, выстраиваемых в процессе компиляции, в которые могут входить:

1. Исходный текст, сохраненный для печати листинга.
2. Таблица имен, содержащая имена и атрибуты переменных.
3. Таблица, содержащая все используемые константы, как целочисленные, так и с плавающей точкой.
4. Дерево разбора, в котором содержится синтаксический анализ программы.
5. Стек, используемый для вызовов процедур внутри компилятора.

В процессе компиляции каждая из первых четырех таблиц постоянно растет. А последняя увеличивается и уменьшается в размерах совершенно непредсказуемым образом.

Сегментация

Каждый сегмент состоит из линейной последовательности адресов от 0 до некоторого максимума. Длина каждого сегмента может иметь любое значение от 0 до максимально разрешенного. Различные сегменты могут быть разной длины, как это обычно и случается. Кроме того, длина сегмента может изменяться в процессе выполнения программы. Длина сегмента стека может увеличиваться при поступлении в него данных и уменьшаться при их извлечении из него.

Сегментированная память дает возможность каждой таблице разрастаться или сужаться независимо от всех остальных таблиц.

